



Guía para conseguir el 100% en el "Hito 4"

El Hito 4 consolida el paso del backend Java desacoplado hacia un **microservicio productivo, persistente y documentado con Spring Boot, PostgreSQL y Docker**. El objetivo de esta etapa es exponer los casos de uso mediante APIs REST semánticas, persistir los datos de forma real en una base de datos relacional contenedorizada y generar contratos técnicos seguros con OpenAPI y Swagger.

Para alcanzar la calificación máxima (**10/10 - Nivel "Excelente"**), el repositorio en GitHub debe cumplir minuciosamente con los tres pilares evaluables de la rúbrica oficial.

Pilar 1: API REST Semántica y Manejador Global de Errores (3 Puntos)

El objetivo: Exponer los controladores web REST en la capa de infraestructura operando bajo rutas semánticas y verbos HTTP correctos, interceptando cualquier excepción de negocio de forma centralizada con `@RestControllerAdvice`.

Checklist para el Nivel "Excelente":

- **Controladores REST Semánticos:** Clases anotadas con `@RestController` y `@RequestMapping("/api/v1/recursos")`. Uso estricto de `@GetMapping` (para consultas con retorno 200 OK) y `@PostMapping` (para creaciones con retorno 201 Created).
- **Cero Trazas Nativas de Servidor:** Queda estrictamente prohibido devolver respuestas 500 Internal Server Error con stacktraces crípticos ante errores de validación o lógica.
- **Interceptor Perimetral Centralizado:** Implementar una clase anotada con `@RestControllerAdvice` que capture las excepciones de negocio (ej. `OutOfStockException`, `ResourceNotFoundException`) mediante `@ExceptionHandler` y devuelva un DTO unificado (`ErrorResponse` o `ApiResponse`) con códigos semánticos (400, 404, 422).

Java

```
// src/main/java/com/tunegocio/infraestructura/web/controller/GlobalExceptionHandler.java
package com.tunegocio.infraestructura.web.controller;
```

```
import com.tunegocio.domain.exception.OutOfStockException;
import com.tunegocio.infraestructura.web.dto.ErrorResponse;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;
import java.time.LocalDateTime;
```

```

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(OutOfStockException.class)
    public ResponseEntity<ErrorResponse> handleOutOfStock(OutOfStockException ex) {
        ErrorResponse error = new ErrorResponse(
            ex.getMessage(),
            "BUSINESS_RULE_VIOLATION",
            LocalDateTime.now()
        );
        return new ResponseEntity<>(error, HttpStatus.UNPROCESSABLE_ENTITY); // 422
    }
}

```

Pilar 2: Virtualización y Persistencia Real con Docker y JPA (3 Puntos)

El objetivo: Sustituir los arreglos o simulaciones en memoria por una base de datos relacional PostgreSQL levantada en un contenedor Docker, mapeando las tablas mediante entidades JPA y repositorios de Spring Data.

Checklist para el Nivel "Excelente":

- **Orquestación con Docker Compose:** Archivo docker-compose.yml (o compose.yml) en la raíz del repositorio, configurando el servicio de PostgreSQL, variables de entorno seguras y un volumen persistente para no perder datos al reiniciar.
- **Mapeo JPA en Infraestructura:** Clases de entidad en infrastructure.persistence decoradas con @Entity, @Table y @Id. Las anotaciones de JPA tienen prohibido residir en las clases de Dominio puro.
- **Repositorios Spring Data JPA:** Interfaces que extiendan de JpaRepository<Entity, ID> para automatizar operaciones CRUD sin sentencias SQL manuales en código duro.

YAML

```

# docker-compose.yml (en la raíz del proyecto)
version: '3.8'
services:
  db:
    image: postgres:16-alpine
    container_name: tunegocio-postgres-db
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: tunegocio_db

```

```
POSTGRES_USER: dev_user
POSTGRES_PASSWORD: SecureDevPassword123
volumes:
  - postgres_data:/var/lib/postgresql/data
```

```
volumes:
  postgres_data:
```

```
Java
//
src/main/java/com/tunegocio/infrastructure/persistence/repository/BookingJpaRepository.java
package com.tunegocio.infrastructure.persistence.repository;

import com.tunegocio.infrastructure.persistence.entity.BookingEntity;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface BookingJpaRepository extends JpaRepository<BookingEntity, String> {
    // Spring Data JPA resuelve las operaciones CRUD de forma nativa
}
```

Pilar 3: Contratos Seguros con Swagger-UI y Perfiles (4 Puntos)

El objetivo: Integrar Springdoc OpenAPI para documentar las rutas y DTOs de forma automatizada, blindando la aplicación mediante perfiles de Spring (@Profile("dev") o application-dev.yml) para asegurar que Swagger quede bloqueado en producción.

Checklist para el Nivel "Excelente":

- **Documentación OpenAPI Activa:** Consola Swagger-UI totalmente funcional y accesible localmente en <http://localhost:8080/swagger-ui.html> bajo el perfil de desarrollo.
- **Anotaciones Técnicas de Contrato:** Decorar controladores con @Tag, @Operation, @ApiResponse y los DTOs con @Schema para autocompletar el botón interactivo "Try it out".
- **Aislamiento Hermético por Perfiles (Ciberseguridad):** Configurar springdoc.swagger-ui.enabled: false y springdoc.api-docs.enabled: false en el perfil general/productivo para eliminar la superficie de ataque fuera del entorno de desarrollo local.

YAML

src/main/resources/application-dev.yml (Perfil Local de Desarrollo)

spring:

datasource:

url: jdbc:postgresql://localhost:5432/tunegocio_db

username: dev_user

password: SecureDevPassword123

driver-class-name: org.postgresql.Driver

jpa:

hibernate:

ddl-auto: update

show-sql: true

springdoc:

api-docs:

path: /api-docs

enabled: true

swagger-ui:

path: /swagger-ui.html

enabled: true

try-it-out-enabled: true

YAML

src/main/resources/application-prod.yml (Perfil Seguro de Producción)

springdoc:

api-docs:

enabled: false # Bloquea la descarga de la especificación técnica

swagger-ui:

enabled: false # Devuelve 404 ante intentos de acceso a la consola web

El Entregable: Un README.md Profesional

El repositorio en GitHub debe actualizarse incluyendo:

1. **Descripción del Microservicio:** Explicación del sistema y su pila tecnológica (Java, Spring Boot, PostgreSQL, Docker, OpenAPI).
2. **Instrucciones para Levantar la Base de Datos:**
Bash
docker compose up -d
3. **Instrucciones para Ejecutar la Aplicación en Modo Desarrollo:**

Bash

`./mvnw spring-boot:run`

4. **Ruta de Documentación y Pruebas de Contratos:**

- Swagger-UI: <http://localhost:8080/swagger-ui.html>
- OpenAPI JSON: <http://localhost:8080/api-docs>

5. **Colección de Pruebas:** Mención o inclusión del archivo de colección Bruno (.bru) o Postman utilizado para auditar los endpoints. (no obligatorio)